project1：TTL 与统计元数据功能实现

0.分组说明

实验允许至多2人一个小组。

小组划分建议按照之前代码报告时的划分。

1. 实验准备

两个实验共用一套base代码，需要在同一份源码中同时实现 TTL 和统计元数据功能，并保证两者协同工作。两套 base 测试分别运行，统一 benchmark 同时评测各自功能及两者的协同正确性。

克隆仓库：

```
git clone --branch master --single-branch \
  https://gitea.shuishan.net.cn/CDMS.XuanZhou.2026Fall.Dase/leveldb_project1
```

进入仓库根目录后，直接修改 `db/` 、 `include/` 等源码。

1.1. 构建与测试

使用 Linux 或 Windows 的 WSL2 环境，安装 CMake 3.22+、支持 C++17 的编译器和 Python 3.9+。依赖源码已随仓库提供，无需初始化 Git 子模块。

根目录运行：

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug \
  -DLEVELDB_BUILD_TESTS=ON -DLEVELDB_BUILD_BENCHMARKS=OFF
cmake --build build -j2
(cd build && ./ttl_test)
(cd build && ./metadata_test)
mkdir -p build/test-data
TEST_TMPDIR="$PWD/build/test-data" ctest --test-dir build --output-on-failure
```

TTL 接口初始返回 `NotSupported` ，统计接口尚未维护计数，因此课程测试和benchmark 中的相关功能测试会失败，需要完成实验后通过。

1.2. benchmark 自测

在仓库根目录运行统一 benchmark，检查 TTL、统计元数据及两者协同工作的正确性和性能：

```
python3 benchmark/runner.py --kind combined --source "$PWD" \
  --output "$PWD/build/evaluation-combined-quick" --profile quick --jobs 2
```

`quick` 用于开发自测；性能测量使用 `--profile standard --matrix` ，每次运行需指定新的输出目录。

开发过程中可以使用 `--kind ttl` 或 `--kind metadata` 单独调试，最终应通过完整的 `combined` 评测。

性能对照的运行方法详见
https://gitea.shuishan.net.cn/CDMS.XuanZhou.2026Fall.Dase/leveldb_project1/src/branch/master/benchmark
<https://gitea.shuishan.net.cn/CDMS.XuanZhou.2026Fall.Dase/leveldb_project1/src/branch/master/benchmark>

3576字

在自己的设备上运行benchmark，并按实验要求补充测试。正式评测由老师在统一机器上完成。

2. 功能一：TTL

2.1. 实验目的

- 理解 LevelDB 的写入、读取、版本管理和 Compaction 流程。
- 掌握数据有效期的存储、判断和回收方法。
- 通过修改源码和编写测试，验证新增功能的正确性与性能。

2.2. 实验要求

在 LevelDB 中实现 TTL 功能，使数据在到期后无法读取，并在 Compaction 中被清理。补全带 TTL 的写入接口，保留原有的普通写入接口：

```
1  Status Put(const WriteOptions& options, const Slice& key,
2             const Slice& value, uint64_t ttl);
3  Status Put(const WriteOptions& options, const Slice& key,
4             const Slice& value);
```

1. **写入有效期。** TTL 的单位为秒。系统使用 `Options::env->NowMicros()` 获取写入时的时间，并保存绝对到期时间；普通 Put 写入的数据不会过期。
2. **过滤过期数据。** 当读取时间达到或超过到期时间时，Get 返回 `NotFound`，迭代器跳过该记录。快照读取使用快照创建时的序列号和时间；未指定快照的迭代器固定自身创建时的视图。
3. **清理过期记录。** Compaction 删除不再需要的过期记录，同时保留活跃快照和迭代器仍需读取的版本。
4. **支持恢复。** 数据刷盘、合并和数据库重开后，系统保留原到期时间，不重新计算有效期。

TTL 为 0 时，系统写入一个立即过期的新版本，并正常分配序列号。时间计算溢出时，接口返回 `InvalidArgument`，不写入数据，也不推进序列号。

新增功能应保持普通读写、删除和 WriteBatch 等原有操作正常工作。

2.3. 实验内容

2.3.1. TTL 功能介绍

TTL（Time To Live，生存时间）用于限制数据的有效期。例如，会话记录和缓存数据可以在指定时间后自动失效。实现时需要同时处理读取过滤和存储回收。

2.3.2. 设计方案

自行选择有效期的编码方式，说明它如何经过 MemTable、WAL 和 SST 保存与恢复。分析读取、快照和 Compaction 应如何使用这些信息。

2.3.3. 实现步骤

1. **修改写入流程。** 计算并保存到期时间，区分普通写入和 TTL 写入。
2. **修改读取流程。** 在 Get 和迭代器中判断有效期，并支持快照视图。
3. **修改 Compaction。** 清理可以回收的过期记录，保护仍被读取视图使用的版本。
4. **完成恢复流程。** 正常重开和进程异常退出后，数据及有效期仍然正确。

提供可重复执行的回收验证方法：停止写入、释放读取视图、刷盘并完成目标 SST 的重写，等待后台任务结束。该方法应能处理最深层 SST，可使用手动合并或测试辅助接口。

2.3.4. 测试与验证

base 已提供的测试

`test/ttl_test.cc` 提供以下两项测试，运行程序为 `ttl_test`：

测试	已有测试检查的内容
`ReadTTL`	测试写入 TTL 为 20 秒的数据，检查到期前读取成功，等待数据过期后读取失败。
`CompactionTTL`	测试等待数据过期后调用 `CompactRange`，检查指定范围的估算数据大小从非零变为零。

需要补充的测试

应在通过 base 测试的基础上，自行补充以下验证：

- 检查普通写入、TTL 写入、覆盖和删除的组合，确认新值过期后不会重新读到旧值。
- 检查零 TTL、时间溢出和恰好到期的情况，并核对快照、Seek 及正反向遍历的结果。
- 检查刷盘、Compaction 和重开后的数据及有效期；回收完成后，核对过期内部记录是否被清除。
- 检查多线程访问和进程异常退出后的恢复。异常退出测试应先确认 `sync=true` 写入成功，再结束进程并重开数据库。

建议使用可控时钟测试时间边界。每个测试应使用独立数据库，并检查读写操作的返回状态；过期读取应返回 `NotFound`。

同时运行公开 benchmark，比较不同数据规模、线程数和过期比例下的写入、读取及回收开销。每组至少运行三次，并在报告中说明测试方法、结果和性能变化。

3. 功能二：统计元数据

3.1. 实验目的

- 理解 LevelDB 的内部记录、序列号和 Compaction 流程。
- 掌握统计信息的维护、一致性查询和持久化恢复方法。
- 分析新增统计功能对读写、内存和启动时间的影响。

3.2. 实验要求

在 LevelDB 中实现统计元数据功能，通过以下接口返回当前数据库的记录总数、去重键数和最新序列号：

```cpp
struct Stats {
  uint64_t total_entries = 0;
  uint64_t distinct_keys = 0;
  uint64_t seq = 0;
};

Status GetCurrentStats(const ReadOptions& options, Stats* stats);
```

字段	统计内容
`total_entries`	系统统计尚未回收的内部记录。同一键的多个版本和删除标记分别计数。
`distinct_keys`	系统对上述记录按用户键去重。已删除但仍有内部记录的键仍然计入。
`seq`	系统返回与统计结果对应的最新已发布序列号，该序列号属于整个数据库。

统计范围包括当前 MemTable、Immutable MemTable 和当前版本中的有效 SST，不重复计算 WAL 副本。新建空数据库的三个字段均为 0。

TTL 记录按上述范围计入统计；已过期但尚未回收的记录仍然计数，实际回收后再扣除相应数量。时间流逝和 Compaction 回收本身不推进序列号。

1. **维护统计。** 系统在写入、覆盖、删除和批量操作后更新统计，在 Compaction 回收记录后扣除相应数量。
2. **保证一致性。** 一次查询返回的三个字段应属于同一时刻的数据库状态，并发操作和 WriteBatch 不能让查询读到不完整的统计结果。
3. **支持恢复。** 数据库重开后，系统应恢复正确的统计。学生可以保存统计信息，也可以根据持久化记录重建统计。

接口始终查询当前状态，忽略 `ReadOptions::snapshot`；`stats` 为空指针时返回 `InvalidArgument`。新增功能应保持 LevelDB 原有操作正常工作，学生需要补充测试验证统计和恢复的正确性。

3.3. 实验内容

3.3.1. 功能介绍

统计信息用于观察数据库的数据规模和版本情况。`total_entries` 反映当前保存的记录数，`distinct_keys` 反映涉及的用户键数，`seq` 标识这组统计对应的写入进度。

例如，在没有发生记录回收时，依次执行：

操作完成后	`total_entries`	`distinct_keys`	`seq`
新建空库	0	0	0
Put("a", "1")	1	1	1
Put("b", "1")	2	2	2
Put("a", "2")	3	2	3
Delete("b")	4	2	4

Compaction 清理旧版本或删除标记后，记录数随实际保留状态更新。

3.3.2. 设计方案

设计用于维护记录数和去重键数的数据结构，说明内存统计、文件统计和全局统计之间的关系。明确状态更新时机，以及查询如何取得一致结果。分析维护开销、查询开销、额外内存和启动代价。

3.3.3. 实现步骤

1. **定义统计结构。** 阅读 base 中的 Stats 和查询接口，设计内部维护方式。
2. **修改写入流程。** 处理 Put、Delete 和 WriteBatch 对记录数、键数及序列号的影响。
3. **修改 Compaction 流程。** 在新状态安装时更新统计，移除已被回收记录的贡献。
4. **实现查询与恢复。** 返回一致的统计结果，并在重启后恢复正确状态。

3.3.4. 测试与验证

base 已提供的测试

`test/metadata_test.cc` 提供以下三项测试，运行程序为 `metadata_test`：

测试	已有测试检查的内容
`BasicCount`	测试依次写入 `k1`、`k2`、`k1`，检查记录总数为 3、去重键数为 2。
`PersistAndRecover`	测试写入数据后关闭并重新打开数据库，检查重开前后的记录总数和去重键数一致。

3576字

`CompactionUpdate`	测试写入 10,000 个不同的键并调用 `CompactRange`，检查记录总数和去重键数仍大于 0。

需要补充的测试

base 测试尚未核对序列号，也没有检查回收后的精确计数。应进一步补充以下验证：

- 对空库、重复写入、删除和 WriteBatch 逐步计算预期结果，并核对三个统计字段。
- 让相同键出现在不同内存表和 SST 中，检查系统是否正确去重；回收旧版本或删除标记后，核对计数是否正确减少。
- 在并发写入、统计查询和 Compaction 期间检查结果，确认一次查询不会混合不同时刻或半个批次的统计。
- 联合验证 TTL 写入、过期、读取视图保护、回收和重开后的统计，核对读取结果与三个统计字段。
- 检查正常重开和进程异常退出后的恢复，并核对三个字段的正确值。异常退出测试应先确认 `sync=true` 写入成功，再结束进程。

预期结果应根据操作记录或实际内部记录独立计算，不能只比较两次接口返回值。每个测试应使用独立数据库，并检查关键操作的返回状态。

同时运行公开 benchmark，比较不同数据规模和线程数下的统计查询开销，以及查询对写入性能的影响。每组至少运行三次，并在报告中说明测试方法、结果和性能变化。

4. 提交要求

4.1. 代码提交

- 在码园创建仓库并提交同时实现 TTL 和统计元数据功能的一份 LevelDB 源码，需能成功编译和运行。码园地址：

https://gitea.shuishan.net.cn/CDMS.XuanZhou.2026Fall.Dase
<https://gitea.shuishan.net.cn/CDMS.XuanZhou.2026Fall.Dase>

- 使用 Git 管理代码，为关键修改添加必要注释。
- **写明姓名、学号、分工、依赖和构建测试命令。**

代码提交截止日期：11月5日

4.2. 实验报告

- 描述设计思路和实现过程。
- 包含测试用例和测试结果截图。
- 分析实验中遇到的问题和解决方案。
- 使用 coding agent 时，说明其参与范围和本人完成的核验。

实验报告提交截止日期：11月5日

5.评分标准

实验报告与汇报 30%

公开Benchmark测试 40%

测试部分	主要内容
TTL 测试	检查有效期、过期读取和读取视图，并测量 TTL 写入、混合读写及过期回收的开销。
统计元数据测试	检查统计接口与重开后的统计结果，并测量统计查询及并发写入时的查询开销。

3576字

联合测试	检查 TTL 写入、过期、回收和恢复时的统计一致性，并测量 TTL 写入期间的统计查询及回收开销。
通用测试	测量普通写入、读取、扫描、重开和并发操作的性能，并执行 LevelDB 原生回归测试。

不公开benchmark测试 30%（正确性、稳定性和性能要求更高）

***不给出的benchmark在提交前有一次额外的测试机会**